

DOKÜMAN: Frontend Case Study (React)

Senaryo: Güvenli Kullanıcı Verisi Yönetimi Arayüzü

Backend'de başarıyla geliştirilen Güvenli Kimlik ve Meslek Bilgileri Yönetimi API'si için zengin özellikli ve performanslı bir kullanıcı arayüzü (UI) geliştirilmesi talep edilmektedir. Bu arayüz, kullanıcıların rollerine göre yetkilendirme, karmaşık veri filtreleme ve kullanıcı dostu bir arayüz ile büyük veri setlerini yönetmesini sağlamalıdır.

Kritik Teknik Gereksinimler (Frontend Mimarisi)

- State Yönetimi:** Uygulamanın durum yönetimi **gerekli görülen yapı (Redux, ContextApi, Zustand, vb.)** kullanılarak yapılmalıdır.
- Auth State:** Kullanıcının JWT (token, rol bilgisi) durumunun ve oturum açma/kapanma akışının yönetilmesi.
- Veri Senkronizasyonu:** Backend'den çekilen tüm listeleme, filtreleme ve sayfalama parametrelerinin (page, size, sort, filtreler) olarak durum yönetiminde tutulması ve güncellenmesi.
- Performans Optimizasyonu:** Büyük listelerle çalışılırken gereksiz re-render'lar kesinlikle engellenmelidir.

Beklenen Görevler ve Özellikler

1. Güvenlik ve Yetkilendirme (Frontend)

- Login Akışı:** Email/kullanıcı adı ve şifre ile giriş yapıldığında, Backend'den dönen **JWT'nin güvenli bir şekilde saklanması**.
- Yetkilendirme Kontrolü:** Kullanıcının rolüne göre (Örn: Yönetici/Normal Kullanıcı) belirli **Router** veya **Komponentlerin** (Örn: Meslek Grubu Ekle butonu) görünürlüğünün koşullu olarak yönetilmesi.

2. Karmaşık Filtreleme Arayüzü (Backend Query Mantiğının Temsili)

- Orijinal Backend uçlarına karşılık gelen, kullanıcıya karmaşık filtreleme imkanı sunan bir UI tasarlanmalıdır. Filtreler, güncellenmeli ve API isteğini tetiklemelidir.
 - Meslek Grubu Filtresi:** Seçilebilir meslek grupları (Dropdown/Multi-Select).
 - Ad/Soyad Filtresi:** Metin girişi ile adında "Ahmet" geçenleri arama özelliği.
 - TCKN Önek Filtresi:** T.C. kimlik numarasının ilk hanelerine göre filtreleme sağlayan bir giriş alanı.

3. Liste ve Sayfalama Yönetimi (UX ve State Senkronizasyonu)

- API'dan dönen verilerin listelendiği bir **Veri Tablosu (Data Grid)** oluşturulmalıdır.
- Sayfalama (Pagination):** Sayfa numarası, sayfa boyutu (size) ve toplam kayıt sayısı (total count) bilgileri tutulmalı ve kullanıcı arayüzünde doğru gösterilmelidir.

- **Sıralama (Sorting):** Tablo başlıklarına tıklanarak (Örn: Ad, Soyad, Kayıt Tarihi) Backend'e uygun `sort` parametresinin gönderilmesi ve bu parametrenin güncel tutulması.

4. Validasyon ve Hata Yönetimi (Kullanıcı Geri Bildirimi)

- **Form Validasyonu:** Kullanıcı yeni bir kayıt eklerken (Örn: E-posta, TCKN) Backend API'sinin beklediği format ve sınırlar doğrulanmalı ve kullanıcıya anlık geri bildirim verilmelidir.
- **Hata Yakalama ve Gösterme:** API'den dönen hatalar (Örn: 400 Bad Request, 404 Not Found, 409 Conflict) kullanıcıyı bilgilendirecek şekilde (Toast, Banner vb.) anlamlı ve kullanıcı dostu bir dille gösterilmelidir.